

Social Tour Guide Robot

Complete Project Documentation

Team T4 — 41069 Robotics Studio 2

Faculty of Engineering and Information Technology, UTS

Version:	0.1.0
ROS Distro:	ROS2 Humble Hawksbill
Language:	Python 3.10
Maintainer:	Team T4 — Karan Sharma, Vinh Anh, Darren PJ, HaydenLaw
Licence:	Apache-2.0
Repository:	https://github.com/caseinova/full_social_tourguide
Date:	May 2026

Contents

1	Project Overview	8
1.1	Purpose	8
1.2	Target Audience	8
1.3	Scope	8
1.4	Technology Stack	8
1.5	High-Level Architecture	9
1.6	Team and Subsystem Responsibilities	9
I	Subsystem 1 — Locomotion	11
2	Project Overview	12
2.1	Purpose	12
2.2	Target Audience	12
2.3	Scope	12
2.4	Technology Stack	12
2.5	High-Level Architecture	12
2.6	Related Projects	13
3	Getting Started	13
3.1	Prerequisites	13
3.2	System Requirements	13
3.3	Installation	13
3.3.1	Step 1 — Install ROS2 Humble	13
3.3.2	Step 2 — Install TurtleBot3 and Nav2	13
3.3.3	Step 3 — Clone and build the workspace	14
3.4	Environment Variables	14
3.5	Verify the Installation	14
3.6	Quick Start	14
4	Configuration	14
4.1	Launch Arguments — <code>pepper_locomotion_llm.launch.py</code>	14
4.2	Node Parameters	14
4.2.1	<code>dock_listener</code>	14
4.2.2	<code>qr_follower</code>	15
4.2.3	<code>tour_manager</code>	15
5	API and Code Reference	15
5.1	Custom Interfaces	15
5.1.1	<code>TspCommand.msg</code>	15
5.1.2	<code>Tours.srv</code>	16
5.1.3	<code>Description.srv</code>	16
5.2	Data Models	16
5.3	Key Functions	16
6	Architecture and Design	16
6.1	Folder Structure	16
6.2	Node Responsibilities and Topics	17

6.2.1	speech_listener	17
6.2.2	tour_manager	17
6.2.3	dock_listener	18
6.2.4	qr_follower	18
6.3	Key Design Decisions	18
7	Usage Guide	18
7.1	Common Use Cases	18
7.2	Command / Voice Reference	19
7.3	Anti-Patterns	19
8	Testing	19
8.1	Running the Locomotion Test Environment	19
8.2	Mock-Mode Integration Test	19
8.3	What Is and Is Not Tested	20
8.4	Testing Expected Behavior Checklist	21
9	Deployment	21
9.1	Build Process	21
9.2	Launch Files	21
9.2.1	pepper_locomotion_llm.launch.py — recommended for integration	21
9.2.2	qr_follower.launch.py	22
9.3	Post-Deployment Verification	22
9.4	Rollback	22
10	Troubleshooting and FAQs	22
10.1	Common Errors	22
10.2	Useful Debug Commands	22
10.3	Log Locations	22
10.4	FAQs	22
11	Maintenance and Contributing	23
11.1	Branching and Git Workflow	23
11.2	Commit Message Format	23
11.3	Adding a New Node	23
11.4	Adding a New Intent / Feature	23
11.5	Updating Dependencies	23
11.6	Versioning	23
12	Security	23
12.1	Credential Management	23
12.2	Known Security Considerations	23
12.3	Vulnerability Reporting	23
13	Changelog and Versioning	24
13.1	Version History	24
II	Subsystem 2 — Gestures and Communication	25
14	Project Overview	26

14.1 Purpose	26
14.2 Target Audience	26
14.3 Scope	26
14.4 Technology Stack	26
14.5 High-Level Architecture	26
14.6 Related Projects	27
15 Getting Started	27
15.1 Prerequisites	27
15.2 System Requirements	27
15.3 Installation	27
15.3.1 Step 1 — Install qi SDK	27
15.3.2 Step 2 — Build the package	27
15.4 Environment Variables	27
15.5 Verify the Installation	27
15.6 Quick Start	27
16 Configuration	28
16.1 Launch Arguments — <code>pepper_real.launch.py</code>	28
16.2 Node Parameters	28
16.2.1 <code>gesture_manager</code>	28
17 API and Code Reference	28
17.1 Custom Interfaces	28
17.2 Data Models	28
17.3 Key Functions	28
18 Architecture and Design	29
18.1 Folder Structure	29
18.2 Node Responsibilities and Topics	29
18.2.1 <code>gesture_manager</code>	29
18.2.2 <code>hri_coordinator_node</code>	29
18.3 Key Design Decisions	30
19 Usage Guide	30
19.1 Common Use Cases	30
19.2 Command / Voice Reference	30
19.3 Anti-Patterns	30
20 Testing	30
20.1 Running the Test Suite	30
20.2 Mock-Mode Integration Test	31
20.3 What Is and Is Not Tested	31
20.4 Manual Testing Checklist	31
21 Deployment	31
21.1 Build Process	31
21.2 Launch Files	31
21.2.1 <code>pepper_real.launch.py</code> — recommended for integration	31
21.2.2 <code>pepper_sim.launch.py</code> — simulation without hardware	31
21.3 Post-Deployment Verification	31

21.4 Rollback	31
22 Troubleshooting and FAQs	31
22.1 Common Errors	31
22.2 Useful Debug Commands	32
22.3 Log Locations	32
22.4 FAQs	32
23 Maintenance and Contributing	32
23.1 Branching and Git Workflow	32
23.2 Commit Message Format	32
23.3 Code Style	32
23.4 Adding a New Node	32
23.5 Adding a New Intent / Feature	33
23.6 Updating Dependencies	33
23.7 Versioning	33
24 Security	33
24.1 Credential Management	33
24.2 Known Security Considerations	33
24.3 Vulnerability Reporting	33
25 Changelog and Versioning	33
25.1 Version History	33
III Subsystem 3 — HMI and Behavior	34
26 Project Overview	35
26.1 Purpose	35
26.2 Target Audience	35
26.3 Scope	35
26.4 Technology Stack	35
26.5 High-Level Architecture	35
26.6 Related Projects	36
27 Getting Started	36
27.1 Prerequisites	36
27.2 System Requirements	36
27.3 Installation	36
27.3.1 Step 1 — Install Python dependencies	36
27.3.2 Step 2 — Install Ollama (optional, for local LLM)	36
27.3.3 Step 3 — Build the package	37
27.4 Environment Variables	37
27.5 Verify the Installation	37
27.6 Quick Start	37
28 Configuration	37
28.1 Launch Arguments — <code>intent_only.launch.py</code>	37
28.2 Node Parameters	38
28.2.1 <code>speech_to_text_node</code>	38

28.2.2	speech_command_node	38
28.2.3	speech_response_node	38
29	API and Code Reference	38
29.1	Custom Interfaces	38
29.2	Data Models	39
29.3	Key Functions	39
30	Architecture and Design	39
30.1	Folder Structure	39
30.2	Node Responsibilities and Topics	40
30.2.1	speech_to_text_node	40
30.2.2	speech_command_node	40
30.2.3	bt_orchestrator_node	40
30.2.4	speech_response_node	40
30.2.5	waypoint_speaker_node	41
30.3	Behaviour Tree	41
30.4	Key Design Decisions	41
31	Usage Guide	42
31.1	Common Use Cases	42
31.2	Command / Voice Reference	42
31.3	Anti-Patterns	42
32	Testing	42
32.1	Running the Test Suite	42
32.2	Mock-Mode Integration Test	42
32.3	What Is and Is Not Tested	43
32.4	Manual Testing Checklist	43
33	Deployment	43
33.1	Build Process	43
33.2	Launch Files	43
33.2.1	intent_only.launch.py — recommended for integration	43
33.2.2	bringup.launch.py	43
33.3	Post-Deployment Verification	43
33.4	Rollback	43
34	Troubleshooting and FAQs	44
34.1	Common Errors	44
34.2	Useful Debug Commands	44
34.3	Log Locations	44
34.4	FAQs	44
35	Maintenance and Contributing	44
35.1	Branching and Git Workflow	44
35.2	Commit Message Format	44
35.3	Code Style	44
35.4	Adding a New Node	45
35.5	Adding a New Intent / Feature	45
35.6	Updating Dependencies	45

35.7 Versioning	45
36 Security	45
36.1 Credential Management	45
36.2 Known Security Considerations	45
36.3 Vulnerability Reporting	45
37 Changelog and Versioning	45
37.1 Version History	45
37.2 Migration Guide — emotion topic rename	46
37.3 Migration Guide — done_talking timing fix	46
IV Subsystem 4 — Pepper Integration	47
38 Project Overview	48
38.1 Purpose	48
38.2 Target Audience	48
38.3 Scope	48
38.4 Technology Stack	48
38.5 High-Level Architecture	48
38.6 Related Projects	48
39 Getting Started	49
39.1 Prerequisites	49
39.2 System Requirements	49
39.3 Installation	49
39.3.1 Step 1 — Obtain qi SDK	49
39.3.2 Step 2 — Configure Pepper IP	49
39.3.3 Step 3 — Build	49
39.4 Environment Variables	49
39.5 Verify the Installation	49
39.6 Quick Start	49
40 Configuration	50
40.1 Launch Arguments — pepper_real.launch.py	50
40.2 Node Parameters	50
40.2.1 gesture_manager (hardware bridge)	50
41 API and Code Reference	50
41.1 Custom Interfaces	50
41.2 Data Models	50
41.3 Key Functions	50
42 Architecture and Design	50
42.1 Folder Structure	50
42.2 Node Responsibilities and Topics	50
42.2.1 gesture_manager (hardware layer)	51
42.2.2 audio_processor_node	51
42.3 Key Design Decisions	51

43 Usage Guide	51
43.1 Common Use Cases	51
43.2 Command / Voice Reference	51
43.3 Anti-Patterns	51
44 Testing	52
44.1 Running the Test Suite	52
44.2 Mock-Mode Integration Test	52
44.3 What Is and Is Not Tested	52
44.4 Manual Testing Checklist	52
45 Deployment	52
45.1 Build Process	52
45.2 Launch Files	52
45.2.1 pepper_real.launch.py — recommended for hardware deployment	52
45.2.2 pepper_sim.launch.py — simulation without Pepper hardware	53
45.3 Post-Deployment Verification	53
45.4 Rollback	53
46 Troubleshooting and FAQs	53
46.1 Common Errors	53
46.2 Useful Debug Commands	53
46.3 Log Locations	53
46.4 FAQs	53
47 Maintenance and Contributing	54
47.1 Branching and Git Workflow	54
47.2 Commit Message Format	54
47.3 Code Style	54
47.4 Adding a New Node	54
47.5 Adding a New Intent / Feature	54
47.6 Updating Dependencies	54
47.7 Versioning	54
48 Security	54
48.1 Credential Management	54
48.2 Known Security Considerations	54
48.3 Vulnerability Reporting	54
49 Changelog and Versioning	55
49.1 Version History	55
A Complete Topic Reference	55

1 Project Overview

1.1 Purpose

The Social Tour Guide Robot is an autonomous museum and exhibition guide built on two physical platforms: a TurtleBot3 Waffle Pi ground vehicle and a SoftBank Pepper humanoid robot. The system accepts spoken natural-language commands from visitors, navigates to labelled waypoints, delivers spoken explanations, and performs socially-aware gestures through Pepper's arms and tablet display.

1.2 Target Audience

This document targets software engineers and roboticists who will deploy, maintain, or extend the system. Readers are assumed to have working knowledge of ROS2 Humble, Python 3, and Linux.

1.3 Scope

What this project does:

- Accepts wake-word-gated voice commands from visitors.
- Classifies intent via a rule-based fast path and an LLM fallback.
- Navigates TurtleBot3 to recorded waypoints using Nav2.
- Delivers waypoint descriptions through local TTS and Pepper's NAOqi speech service.
- Executes socially-expressive gestures on Pepper (wave, listen, explain, nod).
- Supports custom tour planning with a TSP solver and PyQt5 GUI.
- Saves dock and waypoint poses to SQLite databases for persistence.
- Follows QR codes in two modes: direct velocity control and Nav2 pose goals.
- Runs fully in Gazebo simulation before deploying to physical hardware.

What it intentionally does NOT do:

- Perform SLAM during a live tour — a pre-built map is required.
- Control Pepper's locomotion from ROS2 — Pepper navigates independently via NAOqi.
- Manage visitor ticketing, booking, or CMS integration.
- Support multiple simultaneous visitor groups.

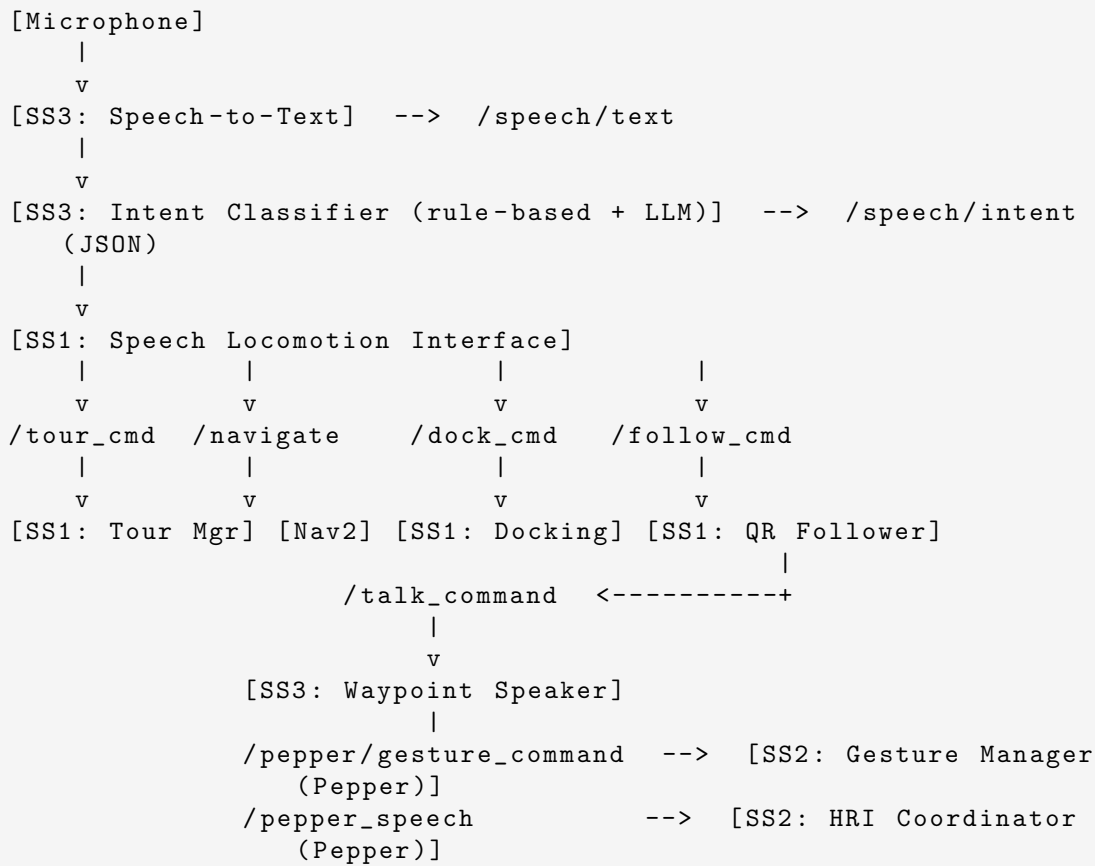
1.4 Technology Stack

Component	Version	Role
ROS2 Humble Hawksbill	LTS 2022	Middleware, inter-node messaging
TurtleBot3 Waffle Pi	—	Ground vehicle with LiDAR and camera
SoftBank Pepper	NAOqi 2.5	Humanoid HRI platform
Nav2	Humble	Autonomous navigation stack
Cartographer	—	SLAM for map building

Component	Version	Role
AMCL	Nav2 built-in	Monte Carlo localisation on pre-built map
faster-whisper	0.10+	Offline speech-to-text (Whisper base model)
Ollama / Groq / OpenAI	—	LLM intent classification back-end
OpenNav docking	Nav2 plugin	Precise docking with pose feedback
OpenCV + cv_bridge	—	QR code detection and camera integration
SQLite3	Python built-in	Waypoint and dock pose persistence
PyQt5	5.x	TSP waypoint selector GUI
qi SDK	NAOqi 2.5	Python bindings for Pepper hardware

1.5 High-Level Architecture

The system operates as a four-layer pipeline on TurtleBot3, with Pepper acting as a side-channel social interface:



1.6 Team and Subsystem Responsibilities

Subsystem	Maintainer	Scope
SS1 — Locomotion	Vinh Anh Nguyen	Speech locomotion interface, Nav2 navigation, waypoint/dock management, QR following, TSP

Subsystem	Maintainer	Scope
SS2 — Gestures & Comm	Hayden Law	Pepper gesture execution, tablet display, audio processing, HRI coordination
SS3 — HMI & Behavior	Karan Sharma	STT, intent classification, TTS response
SS4 — Pepper Integration	Darren Padamittam Jacob	NAOqi bridge, hardware safety, Pepper autonomous navigation

Part I

Subsystem 1 — Locomotion

2 Project Overview

2.1 Purpose

Subsystem 1 handles all ground-vehicle locomotion for the TurtleBot3. It receives high-level intent tokens from Subsystem 3, translates them into Nav2 navigation goals, manages the waypoint and dock databases, executes TSP-optimised multi-stop tours, and provides docking as well as QR-code-based following behaviour.

2.2 Target Audience

Robotics engineers responsible for navigation, map management, and physical robot deployment.

2.3 Scope

What Subsystem 1 does:

- Maps the environment and navigate to any point within it
- Navigates TurtleBot3 to individually named waypoints stored in `tours.db`.
- Executes a full sequential tour through all waypoints on `start_tour` intent.
- Executes a TSP-optimised custom tour subset on `tsp` intent.
- Saves and retrieves dock poses from `docks.db`; Perform docking when a tour ends or when receiving charge command from Subsystem 3.
- Follows a person carrying a QR code in `direct` or `pose` mode.
- Exposes `Tours.srv` and `Description.srv` services for waypoint data.

What it does NOT do:

- Interpret natural language — that is Subsystem 3's responsibility.
- Control Pepper's physical movement.

2.4 Technology Stack

ROS2 Humble, Nav2 (AMCL + DWB controller), Nav2 Simple Commander API, OpenNav docking, OpenCV QR detection, SQLite3, TurtleBot3 Waffle Pi.

2.5 High-Level Architecture

```

/speech/intent (JSON)
  |
  v
[speech_listener] --- NavigationSpeech node
  |         |         |         |
  v         v         v         v
/tour_ /tsp_cmd /dock_ /follow_
command          command  command
  |         |         |         |
  v         v         v         v
[tour_ [TSP+ [dock_ [qr_
manager] GUI] listener] follower]
  |

```

```
v
Nav2 / BasicNavigator
```

2.6 Related Projects

- `turtlebot3_navigation2` — Nav2 launch and parameter files for TurtleBot3.S
- `social_robot_interfaces` — Custom message and service definitions shared across sub-systems.
- `turtlebot_llm_control` (SS3) — Produces `/speech/intent` consumed by `speech_listener`.

3 Getting Started

3.1 Prerequisites

- Ubuntu 22.04 LTS
- ROS2 Humble Hawksbill (full desktop install)
- TurtleBot3 packages: `turtlebot3`, `turtlebot3_simulations`, `turtlebot3_navigation2`
- `opennav_docking` Nav2 plugin
- OpenCV with Python bindings (`python3-opencv`)
- `cv_bridge` ROS2 package

3.2 System Requirements

- 8 GB RAM minimum (16 GB recommended for Gazebo)
- NVIDIA GPU recommended for fast Gazebo rendering
- USB camera or TurtleBot3 camera for QR following

3.3 Installation

3.3.1 Step 1 — Install ROS2 Humble

```
sudo apt update && sudo apt install ros-humble-desktop
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

3.3.2 Step 2 — Install TurtleBot3 and Nav2

```
sudo apt install \
  ros-humble-turtlebot3 \
  ros-humble-turtlebot3-simulations \
  ros-humble-turtlebot3-navigation2 \
  ros-humble-nav2-simple-commander \
  ros-humble-opennav-docking \
  python3-opencv \
  ros-humble-cv-bridge
```

3.3.3 Step 3 — Clone and build the workspace

```
cd ~/
git clone https://github.com/caseinova/full_social_tourguide
cd full_social_tourguide
colcon build --symlink-install
source install/setup.bash
```

3.4 Environment Variables

```
export TURTLEBOT3_MODEL=waffle_pi
export ROS_DOMAIN_ID=30
```

3.5 Verify the Installation

```
ros2 pkg list | grep -E
"tour_manager|speech_locomotion|qr_code|docking"
```

3.6 Quick Start

```
# Simulation with full stack
ros2 launch tour_manager pepper_locomotion_llm.launch.py \
  turtlebot_mode:=real \
  enable_pepper:=true \
  enable_intent:=true
```

4 Configuration

4.1 Launch Arguments — pepper_locomotion_llm.launch.py

Argument	Default	Description
model	waffle_pi	TurtleBot3 model name
lds_model	LDS-01	LDS sensor model for real bringup
turtlebot_mode	sim	sim for Gazebo, real for hardware
use_sim_time	auto	Automatically true in sim mode
nav2_params_file	waffle_pi.yaml	Full path to Nav2 parameter YAML
usb_port	/dev/ttyACMO	OpenCR USB port (real mode only)
namespace	empty	ROS namespace for real TurtleBot3
enable_pepper	true	Launch Pepper HRI real support
enable_intent	true	Launch SS3 intent speech stack

4.2 Node Parameters

4.2.1 dock_listener

Parameter	Default	Description
database_path	docks.db	SQLite file for dock poses
action_name	dock_robot	OpenNav docking action name

Parameter	Default	Description
dock_type	empty	Dock plugin type string
max_staging_time	1000.0	Max time (s) navigating to staging pose
navigate_to_staging_pose	true	Nav2 to staging pose before docking
server_timeout_sec	5.0	Wait for action server (s)
use_dock_id	false	Use OpenNav dock ID vs stored pose
default_dock_id	empty	Fallback dock ID when use_dock_id is true
current_pose_topic	/amcl_pose	Topic for robot pose
global_frame	map	Global reference frame
base_frame	base_link	Robot base frame

4.2.2 qr_follower

Parameter	Default	Description
image_topic	/camera/image_raw	Primary camera image topic
follow_mode	pose	pose (Nav2 goals) or direct (cmd_vel)
qr_size_m	0.16	Physical QR code side length in metres
target_text	empty	Filter QR by decoded text (empty = any)
goal_offset_m	0.45	Stop distance behind QR code
stop_range_m	0.65	Distance at which robot stops
direct_linear_gain	0.45	Proportional gain for cmd_vel linear
direct_angular_gain	1.4	Proportional gain for cmd_vel angular
direct_max_linear_speed	0.14	Cap on forward speed (m/s)
direct_max_angular_speed	0.7	Cap on rotation speed (rad/s)
enabled	false	Start following on node launch
publish_debug_image	false	Publish annotated camera frames

4.2.3 tour_manager

Parameter	Default	Description
tour_description_default	Placeholder name No description provided	Default description inserted when a waypoint has no description

5 API and Code Reference

5.1 Custom Interfaces

5.1.1 TspCommand.msg

```
int64[] waypoints
```

Published on /tsp_command to request a TSP tour over a custom subset of waypoint indices.

5.1.2 Tours.srv

```
int64 idx
---
geometry_msgs/PoseStamped[] tour
```

Service `tour_retrieve`: returns the full list of stored waypoints. The `idx` field is currently unused; pass 0.

5.1.3 Description.srv

```
int64 idx
---
std_msgs/String description
```

Service `retrieve_description`: returns the description string for waypoint at position `idx`. Description format: `<name>|<long text>`.

5.2 Data Models

Waypoints are stored in `tours.db` with the schema:

```
CREATE TABLE tours (
  px REAL, py REAL, pz REAL,
  qx REAL, qy REAL, qz REAL, qw REAL,
  description TEXT
)
```

Dock poses are stored in `docks.db`:

```
CREATE TABLE docks (
  px REAL, py REAL, pz REAL,
  qx REAL, qy REAL, qz REAL, qw REAL
)
```

5.3 Key Functions

`TourManager.retrieve_tour(idx)` queries all rows from `tours.db` and returns them as a list of tuples. The `idx` parameter is accepted but unused; all waypoints are always returned.

`DockingNode.get_nearest_dock_pose()` fetches all dock rows and selects the one closest to the current AMCL pose using Euclidean distance.

`QrFollower.detect_qr(frame)` runs multi-scale, multi-preprocessing detection using OpenCV's `QRCodeDetector` across up to 8 preprocessed image variants per scale.

6 Architecture and Design

6.1 Folder Structure

```
src/
  speech_locomotion_interface/
    speech_locomotion_interface/
```

```

    speech_listener.py      % Intent -> Nav2 / tour / dock / follow
                             bridge
tour_manager/
  tour_manager/
    tour_manager_service.py % Tours.srv + Description.srv +
                             waypoint DB
    tour_saver.py           % Route recording support
  launch/
    pepper_locomotion_llm.launch.py % Master integration launch
    tour_manager_launch.py          % Tour manager only
    locomotion_test.launch.py       % Navigation test launch
docking/
  docking/
    dock_listener.py % OpenNav DockRobot action client
qr_code_follower/
  qr_code_follower/
    qr_follower_node.py % QR-code-based person following
  launch/
    qr_follower.launch.py

```

6.2 Node Responsibilities and Topics

6.2.1 speech_listener

Dir	Topic/Service	Type	Notes
sub	/speech/intent	String	JSON-encoded IntentToken
pub	/tour_command	String	"start" triggers full tour
pub	/tsp_command	TspCommand	Waypoint index array for TSP tour
pub	/dock_command	String	"dock" triggers docking
pub	/follow_command	String	"start" / "stop" QR follow
client	tour_retrieve	Tours.srv	Fetch waypoint list for navigation

6.2.2 tour_manager

Dir	Topic/Service	Type	Notes
srv	tour_retrieve	Tours.srv	Returns all stored PoseStamped waypoints
srv	retrieve_description	Description.srv	Returns description for waypoint index
sub	save_tour	PoseStamped	Appends new waypoint to tours.db
sub	talk_command	String	Extracts index, publishes current tour name
pub	current_tour	String	Name of current destination (for Pepper HRI)

6.2.3 dock_listener

Dir	Topic/Service	Type	Notes
sub	/dock_command	String	Triggers docking sequence
sub	/save_dock_command	String	Saves current pose to docks.db
sub	/amcl_pose	PoseWithCovarianceStamped	Robot localisation pose
action	dock_robot	DockRobot	OpenNav docking action
client			

6.2.4 qr_follower

Dir	Topic/Service	Type	Notes
sub	/camera/image_raw	Image	RGB camera feed
sub	/camera/camera_info	CameraInfo	Intrinsics for pose estimation
sub	follow_command	String	start/stop commands
pub	/qr_follower/status	String	JSON status payload
pub	/cmd_vel	Twist	Direct velocity (direct mode only)
srv	/qr_follower/set_enabled	SetBool	Enable/disable via service
action	navigate_through_poses	NavigateThroughPoses	Pose-mode navigation
client			

6.3 Key Design Decisions

Two QR follow modes. The `direct` mode sends `cmd_vel` commands proportional to QR pixel offset. The `pose` mode transforms the QR 3D pose to map frame via TF2 and queues it as a `NavigateThroughPoses` goal. Direct mode is lower latency but ignores obstacles; pose mode is obstacle-aware but requires TF2 and camera intrinsics.

Nearest-dock selection. When multiple dock poses are stored, `dock_listener` selects the one closest to the current AMCL position. This allows multiple docking stations to coexist in the same map.

SQLite for persistence. Both databases auto-create their tables on first run. Waypoint descriptions are stored inline in the `tours` table as a pipe-delimited string (`name|long text`) so a single SELECT returns everything needed to speak about a waypoint.

7 Usage Guide

7.1 Common Use Cases

Navigate to a waypoint: Say "take me to pillar 3" or "go to pillar three". The `speech_listener` parses the index, calls `tour_retrieve`, and sends the pose to Nav2.

Full tour: Say "start the tour". The `/tour_command` topic triggers the tour controller which iterates through all stored waypoints sequentially.

Custom TSP tour: Say "small tour" or "give me a short tour". The TSP GUI opens; select waypoints and click Start. The 2-opt solver finds the shortest ordering and publishes a TspCommand.

Docking: Say "go home" or "go charge". The /dock_command triggers the docking sequence to the nearest stored dock pose.

QR following: Publish "start" to follow_command or say "follow me". The QR follower activates and tracks the code.

7.2 Command / Voice Reference

See the SS3 section for the complete voice command table. Locomotion responds to intents: navigate, start_tour, tsp, dock, stop_navigation, follow_me.

7.3 Anti-Patterns

- Do not save waypoints while the robot is moving — AMCL pose uncertainty is higher during motion.
- Do not run direct QR mode near obstacles — it publishes raw velocity without collision avoidance.
- Do not set max_staging_time below 60s in large environments — Nav2 may not reach the staging pose in time.

8 Testing

8.1 Running the Locomotion Test Environment

```
cd full_social_tourguide
source install/setup.bash
ros2 launch tour_manager locomotion_test.launch.py
```

8.2 Mock-Mode Integration Test

Begin testing of the subsystem:

For testing the service messages, in a new terminal, type:

```
ros2 service call /tour_retrieve social_robot_interfaces/srv/Tours
    "{idx: 0}"
ros2 service call /retrieve_description \
    social_robot_interfaces/srv/Description "{idx: 0}"
```

To test the message flow of the subsystem, open a new terminal for each of these lines that you want to test:

```
ros2 topic echo /speech/intent
ros2 topic echo /tour_command
ros2 topic echo /tsp_command
ros2 topic echo /dock_command
ros2 topic echo /follow_command
ros2 topic echo /save_tour_command
ros2 topic echo /talk_command
ros2 topic echo /done_talking
```

```
ros2 topic echo /tour_waypoint_order
ros2 topic echo /qr_follower/status
ros2 topic echo /cmd_vel
```

To test waypoint saving, run:

```
ros2 topic pub --once /save_tour_command std_msgs/msg/String
  "{data: save}"
```

To test loading waypoint descriptions into `tours.db`, run:

```
python3 waypoint_info_loader_helper.py
```

Then verify the database:

```
python3 -c "import sqlite3; con=sqlite3.connect('tours.db');
print(con.execute('select rowid, px, py, pz, qx, qy, qz, qw,
description from tours').fetchall())"
```

Then, to test the full tour in a new terminal, use:

```
ros2 topic pub --once /speech/intent std_msgs/msg/String \
  '{data: "{\"intent\":\"start_tour\"}"}'
```

To test the TSP tour, run:

```
ros2 topic pub --once /speech/intent std_msgs/msg/String \
  '{data: "{\"intent\":\"tsp\", \"waypoints\":[0,1,2]}'}'
```

To manually release the waypoint wait, run:

```
ros2 topic pub --once /done_talking std_msgs/msg/String \
  "{data: done_speaking}"
```

To test docking through the intent bridge, run:

```
ros2 topic pub --once /speech/intent std_msgs/msg/String \
  '{data: "{\"intent\":\"dock\"}'}'
```

To test QR following through the intent bridge, run:

```
ros2 topic pub --once /speech/intent std_msgs/msg/String \
  '{data: "{\"intent\":\"follow_me\"}'}'
```

To stop QR following directly, run:

```
ros2 topic pub --once /follow_command std_msgs/msg/String "{data:
  stop}"
```

To test direct navigation to a saved waypoint index, run:

```
ros2 topic pub --once /speech/intent std_msgs/msg/String \
  '{data: "{\"intent\":\"navigate\", \"location\":\"waypoint 0\"}'}'
```

8.3 What Is and Is Not Tested

`/tour_retrieve` returns saved poses from `tours.db`.

`/retrieve_description` returns the expected waypoint description.

`/save_tour_command` saves the robot pose into `tours.db`.

`waypoint_info_loader_helper.py` loads descriptions sequentially and creates extra rows with 0.0 pose values if needed.

Full tour publishes `/tour_command` and visits saved waypoints in index order.

TSP tour publishes `/tsp_command` and `/tour_waypoint_order`.

TalkAtWaypoint publishes `/talk_command` at each reached waypoint.

Navigation resumes after `/done_talking` receives `done_speaking`.

Dock listener receives `/dock_command` and completes docking in simulation.

QR follower activates on `follow_me` and publishes `/cmd_vel`.

QR follower publishes zero velocity within 0.5 s if no fresh QR image is processed.

8.4 Testing Expected Behavior Checklist

Nav2 reaches a saved waypoint within 10 s in the hexagonal arena.

Full tour visits all 9 pillars in index order.

Dock listener navigates to the staging pose and completes the docking sequence.

QR follower activates on "start" and stops within 0.65 m of the code.

TSP GUI opens on "small tour" intent and publishes the selected waypoints.

9 Deployment

9.1 Build Process

```
cd full_social_tourguide
colcon build --symlink-install --packages-select \
  social_robot_interfaces speech_locomotion_interface \
  tour_manager docking qr_code_follower
source install/setup.bash
```

9.2 Launch Files

9.2.1 pepper_locomotion_llm.launch.py — recommended for integration

Launches TurtleBot3 (sim or real), Nav2, tour manager, optional Pepper HRI, and optional SS3 intent stack in a single command.

```
# Simulation
ros2 launch tour_manager pepper_locomotion_llm.launch.py \
  turtlebot_mode:=sim enable_pepper:=false

# Real robot with Pepper
ros2 launch tour_manager pepper_locomotion_llm.launch.py \
  turtlebot_mode:=real enable_pepper:=true
```

9.2.2 qr_follower.launch.py

```
ros2 launch qr_code_follower qr_follower.launch.py \
  follow_mode:=pose enabled:=true
```

9.3 Post-Deployment Verification

```
ros2 topic list | grep -E "tour|dock|follow|qr"
ros2 service list | grep -E "tour_retrieve|retrieve_description"
ros2 topic echo /qr_follower/status --once
```

9.4 Rollback

Restore a previous `tours.db` or `docks.db` from backup. Both files are created in the working directory where the node was launched.

10 Troubleshooting and FAQs

10.1 Common Errors

Symptom	Cause / Fix
<code>tour_retrieve</code> service not available	<code>tour_manager</code> node not running. Start <code>tour_manager.launch.py</code> .
Nav2 goal rejected	Map not loaded or AMCL not localised. Check <code>/amcl_pose</code> topic.
No QR detected	Poor lighting or QR too small. Enable <code>publish_debug_image</code> to inspect the frame.
Docking times out	Staging pose too far. Reduce <code>max_staging_time</code> or resave the dock pose.
Empty tour list	<code>tours.db</code> is empty. Record waypoints first via <code>save_tour</code> topic.

10.2 Useful Debug Commands

```
ros2 topic echo /speech/intent
ros2 topic echo /tour_command
ros2 topic echo /qr_follower/status
ros2 service call /tour_retrieve \
  social_robot_interfaces/srv/Tours "{idx: 0}"
```

10.3 Log Locations

```
~/ros/log/latest/ # ROS2 node logs
tours.db          # Waypoint database (working directory)
docks.db          # Dock database (working directory)
```

10.4 FAQs

How do I add a waypoint? Publish the desired pose on the `save_tour` topic while the robot is at that location, or drive the robot to the spot and use the tour recording interface.

How do I rename a waypoint? Edit the description column in `tours.db` directly using the SQLite3 CLI.

11 Maintenance and Contributing

11.1 Branching and Git Workflow

Use feature branches off `main`. Name branches `feat/<short-description>` or `fix/<short-description>`.

11.2 Commit Message Format

```
<type>(<scope>): <short description>

type: feat | fix | refactor | test | docs
scope: ss1 | ss2 | ss3 | ss4 | interfaces
```

11.3 Adding a New Node

Register the new executable in `setup.py` under `console_scripts`, add it to the relevant launch file, and add its topic table to this document.

11.4 Adding a New Intent / Feature

Add the new intent to `speech_listener.py`'s `intent_callback_` method and publish to the appropriate command topic.

11.5 Updating Dependencies

Update `package.xml` with the new `<depend>` tag, then rebuild with `colcon build`.

11.6 Versioning

Increment the version in `package.xml` following SemVer: MAJOR for breaking interface changes, MINOR for new features, PATCH for bug fixes.

12 Security

12.1 Credential Management

No credentials are used in Subsystem 1. The SQLite databases are local files with no authentication.

12.2 Known Security Considerations

The `/dock_command` and `/tour_command` topics accept any publisher. In a multi-robot or multi-operator environment, restrict topic access via ROS2 DDS security policies.

12.3 Vulnerability Reporting

Report issues via the GitHub repository issue tracker.

13 Changelog and Versioning

13.1 Version History

Version	Date	Changes
0.1.0	May 2026	Initial release. Nav2 waypoint navigation, full tour, TSP solver, OpenNav docking, dual-mode QR follower, SQLite persistence. Added: <code>follow_command</code> topic for QR-based person following. Added: Nearest-dock selection in <code>dock_listener</code> .

Part II

Subsystem 2 — Gestures and Communication

14 Project Overview

14.1 Purpose

Subsystem 2 manages all human-facing interaction on the Pepper robot: gesture execution via NAOqi, tablet display content, audio amplitude monitoring, human presence detection, and event-driven HRI coordination. It bridges the ROS2 ecosystem to Pepper's proprietary NAOqi SDK.

14.2 Target Audience

Engineers integrating ROS2 with NAOqi-based robots. Familiarity with NAOqi 2.5 and the qi Python SDK is assumed.

14.3 Scope

What Subsystem 2 does:

- Executes 5 named gestures on Pepper's arms via `ALMotion.angleInterpolation`.
- Displays exhibit content on Pepper's tablet.
- Monitors audio amplitude and triggers listening gestures when speech is detected.
- Detects human presence and triggers a wave-hello gesture.
- Coordinates the full explanation flow: exhibit trigger → gesture + speech + tablet → reset on done.talking.

What it does NOT do:

- Drive Pepper's wheels — Pepper navigates independently.
- Transcribe speech — that is Subsystem 3's responsibility.
- Manage TurtleBot3 navigation.

14.4 Technology Stack

ROS2 Humble, SoftBank Pepper (NAOqi 2.5), qi Python SDK, NumPy, Python threading.

14.5 High-Level Architecture

```

/pepper/gesture_command (String)
|
v
[gesture_manager] ---> NAOqi ALMotion.angleInterpolation

/current_tour (String)
/human_present (String)
/done_talking (String) [hri_coordinator_node]
/audio/audio (UInt8MultiArray) |
                                +-> /pepper/spoken_words
                                +-> /pepper/gesture_command
                                +-> /pepper/explain_exhibit

```

14.6 Related Projects

- `turtlebot_11m_control` (SS3) — Publishes `/pepper/gesture_command` and `/done_talking`.
- `tour_manager` (SS1) — Publishes `/current_tour` when a new waypoint is reached.

15 Getting Started

15.1 Prerequisites

- SoftBank Pepper robot with NAOqi 2.5 and network connectivity.
- qi Python SDK (NAOqi Python bindings) installed on the ROS2 host.
- Pepper and the ROS2 host on the same LAN segment.

15.2 System Requirements

- Python 3.6+ (qi SDK constraint)
- NumPy (`pip install numpy`) for audio amplitude computation

15.3 Installation

15.3.1 Step 1 — Install qi SDK

```
pip install qi
# or install the NAOqi Python SDK from SoftBank robotics portal
```

15.3.2 Step 2 — Build the package

```
colcon build --packages-select pepper_hri
source install/setup.bash
```

15.4 Environment Variables

No environment variables are required. Pepper's IP address is configured via the `pepper_ip` node parameter.

15.5 Verify the Installation

```
ros2 pkg list | grep pepper_hri
ros2 run pepper_hri gesture_manager &
ros2 topic pub /pepper/gesture_command std_msgs/msg/String \
  "data: 'wave_hello'" --once
```

15.6 Quick Start

```
# Real Pepper
ros2 launch pepper_hri pepper_real.launch.py \
  start_gestures:=true start_hri:=true

# Simulation (no Pepper hardware)
ros2 launch pepper_hri pepper_sim.launch.py
```

16 Configuration

16.1 Launch Arguments — `pepper_real.launch.py`

Argument	Default	Description
<code>params_file</code>	<code>pepper_real.yaml</code>	YAML file with node parameters
<code>start_tablet</code>	<code>true</code>	Launch tablet builder node
<code>start_hri</code>	<code>true</code>	Launch HRI coordinator node
<code>start_gestures</code>	<code>true</code>	Launch NAOqi gesture manager
<code>start_audio</code>	<code>false</code>	Launch audio processor (configure IPs first)

16.2 Node Parameters

16.2.1 `gesture_manager`

Parameter	Default	Description
<code>pepper_ip</code>	<code>192.168.1.100</code>	Pepper's LAN IP address
<code>naoqi_port</code>	<code>9559</code>	NAOqi TCP port
<code>enable_stiffness</code>	<code>true</code>	Call <code>ALMotion.wakeUp()</code> on startup

17 API and Code Reference

17.1 Custom Interfaces

No custom interfaces are defined in Subsystem 2. All topics use `std_msgs/String` or `std_msgs/UInt8MultiArray`.

17.2 Data Models

Gesture table. Five gestures are defined in `gesture_manager.py` as Python dictionaries mapping joint names to angle sequences:

Name	Description
<code>default</code>	Neutral standing pose, arms at sides
<code>listen</code>	Head tilted, right arm raised in attentive listening position
<code>nod</code>	Head pitch oscillates 4 times (acknowledgement)
<code>wave_hello</code>	Right arm raises and wrist oscillates 4 times
<code>explain</code>	Both arms extend forward in presentation stance, looped every 4.5 s

Exhibit database. `hri_coordinator_node` holds an in-memory dictionary of 6 known exhibits, each with a path to a `.txt` file containing the spoken explanation text.

17.3 Key Functions

`GestureManager.execute_gesture(command)` locks `motion_lock` and calls `ALMotion.angleInterpolation` with the joint angle sequences and timing arrays. The lock prevents concurrent gestures.

`HRICoordinatorNode.start_explanation()` loads the text file for the current exhibit, publishes it to `/pepper/spoken_words`, publishes the `explain` gesture, and sets `is_explaining = True`.

`HRICoordinatorNode.done_talking_callback()` resets the explaining state and returns Pepper to the default pose when "done_speaking" arrives on `/done_talking`.

18 Architecture and Design

18.1 Folder Structure

```
src/Pepper_HRI/
  gestures/
    gesture_manager.py      % NAOqi gesture execution node
    gesture_manager_sim.py  % Simulated version (logs instead of
                           NAOqi)
    gesture_manager.launch.py
    default_pose.py        % Pose definition module
    explain_gesture.py      % Explain gesture definition
    listening_gesture.py    % Listening gesture definition
    nodding_gesture.py      % Nodding gesture definition
    wave_hello_gesture.py   % Wave hello gesture definition
  audio/
    audio_processor_node.py % Pepper microphone bridge
  tablet_assets/
    builder.py              % Tablet web server builder
  HRI_coordinator.py        % Event-driven HRI coordinator node
  pepper_real.launch.py     % Launch for real Pepper
  pepper_sim.launch.py      % Launch for simulation
  bootup.launch.py          % Minimal startup launch
```

18.2 Node Responsibilities and Topics

18.2.1 gesture_manager

Dir	Topic	Type	Notes
sub	<code>/pepper/gesture_command</code>	String	Gesture name string

18.2.2 hri_coordinator_node

Dir	Topic	Type	Notes
sub	<code>/current_tour</code>	String	Exhibit name from <code>tour_manager</code>
sub	<code>/audio/audio</code>	UInt8MultiArray	Raw audio from Pepper mic
sub	<code>/human_present</code>	String	Human detection signal
sub	<code>/done_talking</code>	String	"done_speaking" terminates explanation
pub	<code>/pepper/spoken_words</code>	String	Text to speak on Pepper
pub	<code>/pepper/gesture_command</code>	String	Gesture to execute
pub	<code>/pepper/explain_exhibit</code>	String	Exhibit name for tablet display

18.3 Key Design Decisions

Explain gesture looping. The `explain` gesture is driven by a 4.5s timer rather than a single trigger. This keeps Pepper visually engaged throughout a potentially long explanation without requiring the speaker node to issue repeated commands.

Event-driven explanation termination. Rather than using a fixed timer, `hri_coordinator_node` terminates the explanation only when `"done_speaking"` arrives on `/done_talking`. This handles variable-length explanations without timing drift.

Graceful NAOqi reconnection. `GestureManager.start_gesture()` attempts to reconnect to NAOqi if `ALMotion` is `None` before skipping a gesture command. This recovers from transient network interruptions.

19 Usage Guide

19.1 Common Use Cases

Trigger a gesture:

```
ros2 topic pub /pepper/gesture_command std_msgs/msg/String \
  "data: 'wave_hello'" --once
```

Trigger an explanation:

```
ros2 topic pub /current_tour std_msgs/msg/String \
  "data: 'walking_robot_exhibit'" --once
```

Simulate human presence:

```
ros2 topic pub /human_present std_msgs/msg/String \
  "data: 'true'" --once
```

19.2 Command / Voice Reference

Gestures are triggered by ROS2 topic messages, not voice commands directly. Subsystem 3's `waypoint_speaker_node` publishes `"explain"` automatically when describing a waypoint.

19.3 Anti-Patterns

- Do not publish an unknown gesture name — the node logs a warning and ignores it.
- Do not set `enable_stiffness:=false` if Pepper's motors are off — gestures will fail silently.
- Do not start `start_audio:=true` without first setting the correct Pepper IP in `audio_processor_node.py`.

20 Testing

20.1 Running the Test Suite

No automated tests are currently defined for `pepper_hri`. Use manual testing procedures.

20.2 Mock-Mode Integration Test

Run `gesture_manager_sim.py` as a drop-in replacement. It logs gesture commands to the console instead of calling NAOqi.

20.3 What Is and Is Not Tested

NAOqi hardware calls are not covered by automated tests. All gesture sequences have been validated on physical Pepper hardware.

20.4 Manual Testing Checklist

Each of the 5 gesture names triggers the correct Pepper pose.

Human presence detection triggers `wave_hello` within one message cycle.

Explanation flow: exhibit trigger → explain gesture starts → text published → `done_speaking` resets to default.

Gesture manager reconnects after Pepper network dropout.

21 Deployment

21.1 Build Process

```
colcon build --packages-select pepper_hri
source install/setup.bash
```

21.2 Launch Files

21.2.1 `pepper_real.launch.py` — recommended for integration

```
ros2 launch pepper_hri pepper_real.launch.py \
  start_gestures:=true start_hri:=true start_tablet:=true
```

21.2.2 `pepper_sim.launch.py` — simulation without hardware

```
ros2 launch pepper_hri pepper_sim.launch.py
```

21.3 Post-Deployment Verification

```
ros2 topic list | grep pepper
ros2 topic echo /pepper/gesture_command
```

21.4 Rollback

No persistent state is modified by Subsystem 2. Restart the nodes to reset Pepper's posture.

22 Troubleshooting and FAQs

22.1 Common Errors

Symptom	Cause / Fix
Could not connect to Pepper NAOqi	Wrong IP or Pepper not on LAN. Check <code>pepper_ip</code> parameter.
Gesture ignored	Unknown gesture name. Valid names: <code>default</code> , <code>listen</code> , <code>nod</code> , <code>wave_hello</code> , <code>explain</code> .
Explanation never terminates	<code>/done_talking</code> not published. Check SS3 <code>speech_response_node</code> and <code>waypoint_speaker_node</code> .
Audio processor fails to start	Pepper IP not configured in <code>audio_processor_node.py</code> . Edit the file before launching.

22.2 Useful Debug Commands

```
ros2 topic echo /pepper/gesture_command
ros2 topic echo /done_talking
ros2 topic echo /human_present
```

22.3 Log Locations

```
~/ros/log/latest/ # ROS2 node logs
```

22.4 FAQs

How do I add a new exhibit? Add an entry to `exhibit_database` in `HRI_coordinator.py` with the exhibit name as key and the text file path as value.

How do I add a new gesture? Add a joint-angle dictionary to `GESTURES` and a timing array to `GESTURE_DURATIONS` in `gesture_manager.py`.

23 Maintenance and Contributing

23.1 Branching and Git Workflow

Same as the project-wide workflow: feature branches off `main`, prefixed `feat/` or `fix/`.

23.2 Commit Message Format

```
<type>(ss2): <short description>
```

23.3 Code Style

PEP8. Use `flake8` before committing.

23.4 Adding a New Node

Register the executable in `setup.py`, add to a launch file, document topic table in this section.

23.5 Adding a New Intent / Feature

Add the new exhibit to `HRI_coordinator.py`'s `exhibit_database` and provide the corresponding text file under `tablet_assets/`.

23.6 Updating Dependencies

Update `package.xml` and rebuild.

23.7 Versioning

Follow SemVer; increment MINOR for new gestures, PATCH for bug fixes.

24 Security

24.1 Credential Management

No credentials are used. Pepper connectivity is unauthenticated NAOqi over TCP on the local network.

24.2 Known Security Considerations

`/pepper/gesture_command` is unauthenticated. Restrict access on public networks via ROS2 DDS security.

24.3 Vulnerability Reporting

Report issues via the GitHub repository issue tracker.

25 Changelog and Versioning

25.1 Version History

Version	Date	Changes
0.1.0	May 2026	Initial release. Five-gesture library, event-driven HRI coordinator, NAOqi bridge, tablet display, audio amplitude monitoring. Added: <code>done_talking</code> event-driven explanation termination. Breaking: Removed fixed-duration timing from explanation loop — now event-driven only.

Part III

Subsystem 3 — HMI and Behavior

26 Project Overview

26.1 Purpose

Subsystem 3 is the cognitive core of the Social Tour Guide Robot. It captures spoken visitor utterances via offline faster-whisper ASR, classifies intent through a two-stage pipeline (rule-based fast path, then LLM), orchestrates all robot behaviours through a custom Python Behaviour Tree, and generates spoken TTS responses back to the visitor and to Pepper.

26.2 Target Audience

Software engineers working on natural-language HRI, LLM integration, and robot behaviour design.

26.3 Scope

What Subsystem 3 does:

- Offline speech transcription using faster-whisper (no cloud dependency).
- Wake-word detection with a configurable awake window.
- 18-intent rule-based parser covering all navigation, touring, and control commands.
- LLM intent classification fallback (Ollama/Groq/OpenAI) with conversational history.
- Personality-aware natural language response generation.
- Custom Python BT with 6 branches: emergency stop, touring, TSP, navigation, following, interruption.
- TTS responses via `spd-say` / `espeak` / `say`, and forwarded to Pepper NAOqi.
- PyQt5 GUI for TSP waypoint selection.
- Emotion publishing for Pepper gesture coordination.
- Waypoint description retrieval and TTS with gesture synchronisation.

What it does NOT do:

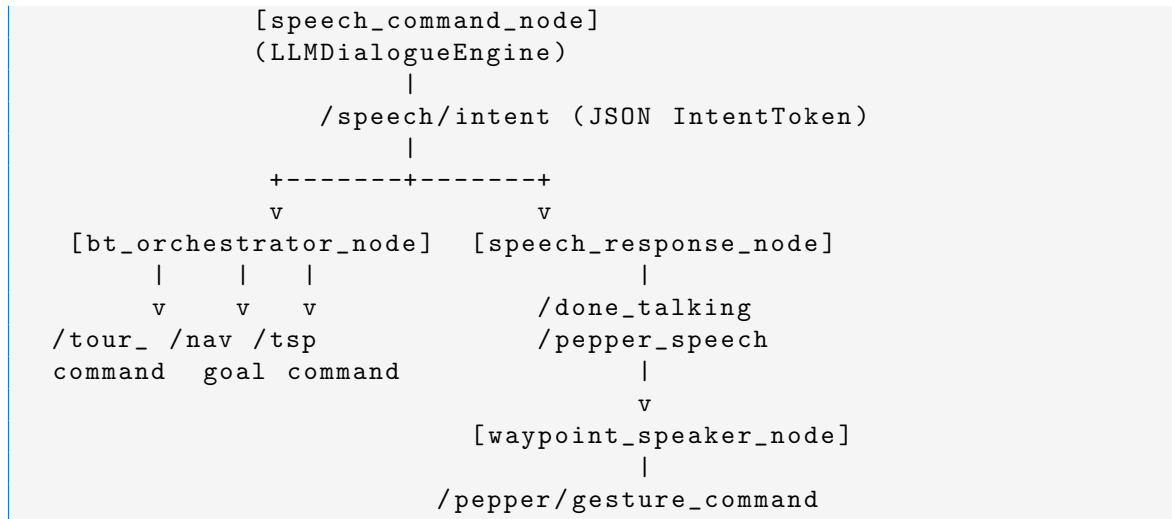
- Implement Nav2 itself — that is SS1's responsibility.
- Control Pepper hardware directly — that is SS2/SS4's responsibility.
- Perform SLAM or map building.

26.4 Technology Stack

ROS2 Humble, faster-whisper (Whisper base model), Python speech_recognition, Ollama / Groq / OpenAI, PyQt5, Python threading.

26.5 High-Level Architecture

```
[Microphone] --> [speech_to_text_node]
                  |
                  /speech/text
                  |
                  v
```



26.6 Related Projects

- `social_robot_interfaces` — `Description.srv` consumed by `waypoint_speaker_node`.
- `tour_manager` (SS1) — Provides `retrieve_description` service.
- `pepper_hri` (SS2/SS4) — Consumes `/pepper_speech` and `/pepper/gesture_command`.

27 Getting Started

27.1 Prerequisites

- Python packages: `faster-whisper`, `speech_recognition`, `pyaudio`, `openai` or `groq`
- `spd-say`, `espeak`, or macOS `say` for local TTS
- Ollama running locally on port 11434 (for default LLM provider)
- PyQt5 (`sudo apt install python3-pyqt5`)

27.2 System Requirements

- 4 GB RAM for `faster-whisper` base model
- Microphone connected or accessible at the configured device index

27.3 Installation

27.3.1 Step 1 — Install Python dependencies

```

pip install faster-whisper openai groq speechrecognition pyaudio
sudo apt install python3-pyqt5 spd-say

```

27.3.2 Step 2 — Install Ollama (optional, for local LLM)

```

curl -fsSL https://ollama.ai/install.sh | sh
ollama pull qwen2.5-coder:latest

```

27.3.3 Step 3 — Build the package

```
colcon build --packages-select turtlebot_llm_control
source install/setup.bash
```

27.4 Environment Variables

Variable	Purpose
GROQ_API_KEY	API key for Groq cloud LLM
OPENAI_API_KEY	API key for OpenAI cloud LLM

Alternatively, place the key in `turtlebot_llm_control/api.key` (this file is in `.gitignore`).

Warning: Never commit API keys to the repository. Use environment variables or the `api.key` file.

27.5 Verify the Installation

```
python3 -c "import faster_whisper; print('whisper ok')"
ros2 run turtlebot_llm_control speech_to_text_node
```

27.6 Quick Start

```
# Intent pipeline only (no Nav2 required)
ros2 launch turtlebot_llm_control intent_only.launch.py \
  enable_microphone:=true \
  llm_provider:=ollama \
  llm_model:=qwen2.5-coder:latest
```

28 Configuration

28.1 Launch Arguments — `intent_only.launch.py`

Argument	Default	Description
<code>enable_microphone</code>	<code>true</code>	Capture live audio from mic
<code>enable_llm</code>	<code>true</code>	Enable LLM fallback classification
<code>llm_provider</code>	<code>ollama</code>	<code>ollama</code> , <code>groq</code> , or <code>openai</code>
<code>llm_model</code>	<code>qwen2.5-coder:latest</code>	Model identifier for the selected provider
<code>llm_api_key_path</code>	<code>empty</code>	Path to API key file
<code>energy_threshold</code>	<code>150</code>	Mic capture sensitivity (lower = more sensitive)
<code>require_wake_word</code>	<code>true</code>	Gate commands behind wake word
<code>wake_command_window_seconds</code>	<code>45.0</code>	Awake window after wake word (s)
<code>phrase_time_limit</code>	<code>5.0</code>	Max utterance duration (s)
<code>mute</code>	<code>false</code>	Suppress local TTS output
<code>enable_tsp_gui</code>	<code>true</code>	Launch TSP waypoint selector GUI

Argument	Default	Description
enable_emotion	true	Launch emotion publisher node
enable_waypoint_speaker	true	Launch waypoint description TTS node

28.2 Node Parameters

28.2.1 speech_to_text_node

Parameter	Default	Description
enable_microphone	true	Enable live mic capture
mic_device_index	-1	Mic device index; -1 = system default
energy_threshold	150	Capture sensitivity (SpeechRecognition)
dynamic_energy_threshold	false	Auto-adjust threshold to ambient noise
calibrate_ambient_noise	false	Calibrate threshold on startup
phrase_time_limit	5.0	Max phrase duration (s)
require_wake_word	true	Wake-word gate
wake_command_window_seconds	45.0	Awake window duration (s)

28.2.2 speech_command_node

Parameter	Default	Description
enable_llm	true	Enable LLM intent classification
llm_provider	ollama	LLM provider name
llm_model	qwen2.5-coder:latest	Model identifier
llm_api_key_path	empty	Path to API key file

28.2.3 speech_response_node

Parameter	Default	Description
mute	false	Skip local TTS; publish done_talking immediately

29 API and Code Reference

29.1 Custom Interfaces

IntentToken is the central data model, serialised as JSON and published on /speech/intent:

```
@dataclass
class IntentToken:
    intent: str          # e.g. "navigate", "start_tour"
    label: str = ""      # route label for recording intents
    location: str = ""   # normalised location for navigate/explain
    utterance: str = ""  # original spoken text
    response: str = ""   # robot's spoken reply
    metadata: dict = field(default_factory=dict)
```

29.2 Data Models

Allowed intents:

Intent	Trigger
no_action	Conversational input not requiring robot motion
is_alive	Wake/alive check: "hey pepper", "are you there"
stop	Immediate halt: "stop", "halt", "freeze", "abort"
stop_navigation	Cancel current Nav2 goal only
navigate	Navigate to named waypoint
pause	Pause current task
resume	Resume paused task
manual_override_on	Enable teleop mode
manual_override_off	Disable teleop mode
start_tour	Execute full sequential tour
tsp	Open TSP GUI for custom tour subset
dock	Navigate to charging station
follow_me	Activate QR-code following behaviour
start_recording	Begin route recording
stop_recording	Stop route recording
save_route	Save recorded route with label
replay_route	Replay saved route
explain	Request verbal description of a location

29.3 Key Functions

`LLMDialogueEngine.resolve_token(utterance)` is the main entry point. It first attempts `parse_utterance()` (rule-based), then `generate_llm_token()` (LLM), then `fallback_chat_token()` (static responses).

`parse_utterance(text)` in `speech_parser.py` normalises the input and matches against ordered conditional branches. Emergency stop is checked first; navigation commands last.

`normalize_location_name(text)` converts spoken location names to canonical form: "pillow three" and "below 3" both resolve to `pillar_3`.

30 Architecture and Design

30.1 Folder Structure

```
src/turtlebot_LLM_control/
  turtlebot_llm_control/
    models.py           % IntentToken dataclass
    wake_word.py        % Wake word detection utilities
    speech_parser.py     % Rule-based intent parser
    llm_dialogue.py      % LLMDialogueEngine
    audio_utils.py       % TTS wrapper (spd-say/espeak/say)
    predefined_commands.py % Predefined command fast path
    behavior_tree.py     % BT node types (BTNode, Sequence,
                          Fallback)
    bt_orchestrator_node.py % Main BT orchestrator ROS2 node
```



```

speech_to_text_node.py      % faster-whisper STT node
speech_command_node.py     % Intent classification node
speech_response_node.py    % TTS response node
waypoint_speaker_node.py   % Waypoint description TTS node
emotion_node.py            % Emotion signal publisher
tsp_gui_node.py            % PyQt5 TSP waypoint selector
tour_intent_bridge_node.py % Tour teleop trigger bridge
tour_recording_manager.py  % Route recording manager
tour_teleop_session.py     % Teleop route session
sim_initial_pose_node.py   % Simulation pose initialiser
launch/
  intent_only.launch.py    % Recommended for integration
  bringup.launch.py        % Full system bringup
  sim_pillar_nav_demo.launch.py

```

30.2 Node Responsibilities and Topics

30.2.1 speech_to_text_node

Dir	Topic	Type	Notes
pub	/speech/text	String	Transcribed utterance

30.2.2 speech_command_node

Dir	Topic	Type	Notes
sub	/speech/text	String	Raw transcription in
pub	/speech/intent	String	JSON-encoded IntentToken
pub	/speech/response	String	Robot's spoken response text

30.2.3 bt_orchestrator_node

Dir	Topic	Type	Notes
sub	/speech/intent	String	Intent JSON consumed by BT
pub	/tour_command	String	Start full tour
pub	/tsp_command	TspCommand	Custom TSP tour
pub	/dock_command	String	Trigger docking
pub	/follow_command	String	Toggle QR following
pub	/navigate_to_pose	PoseStamped	Direct Nav2 goal

30.2.4 speech_response_node

Dir	Topic	Type	Notes
sub	/speech/response	String	Text to speak
pub	/speech/tts_active	Bool	TTS busy signal

Dir	Topic	Type	Notes
pub	/pepper_speech	String	Forwarded text to Pepper TTS
pub	/done_talking	String	"done_speaking" after TTS ends

30.2.5 waypoint_speaker_node

Dir	Topic/Service	Type	Notes
sub	/talk_command	String	Integer index of waypoint to describe
client	retrieve_description	Description.srv	Fetches description from tour_manager
pub	/pepper/gesture_command	String	"explain" before TTS starts
pub	/robot/emotion	String	JSON emotion payload
pub	/done_talking	String	"done_speaking" after TTS ends
pub	/done_speaking	String	Duplicate signal for compatibility

30.3 Behaviour Tree

The BT is ticked on every new intent token. The root is a `FallbackNode` over six branches:

Branch	Trigger / Action
Emergency Stop	<code>stop</code> intent — cancels all Nav2 tasks and resets state immediately
Touring	<code>start_tour</code> intent — starts sequential waypoint tour
TSP	<code>tsp</code> intent — opens GUI and executes optimised subset tour
Navigation	<code>navigate</code> intent — sends single pose goal to Nav2
Following	<code>follow_me</code> intent — activates QR follower
Interruption	<code>pause/resume/manual_override</code> — pauses or resumes active tasks

30.4 Key Design Decisions

Rule-based fast path. Navigation-critical commands are matched by `parse_utterance()` before the LLM is consulted. This eliminates approximately 500 ms of LLM latency for safety-relevant inputs such as stop and emergency halt.

Greedy LLM decoding. The intent classification call uses `temperature=0.5` and `max.tokens=150` to minimise response time and maximise determinism.

Non-blocking TTS. Both `speech_response_node` and `waypoint_speaker_node` run TTS in daemon threads. This prevents the TTS subprocess from blocking the ROS2 executor and ensures `/done_talking` is only published after speech actually finishes.

Location normalisation. The `normalize_location_name()` function maps acoustic confusables (“pillow”, “below”) to the canonical “pillar” string before database lookup, preventing missed navigations from mis-transcription.

31 Usage Guide

31.1 Common Use Cases

Single navigation: Say "take me to pillar 5".

Full tour: Say "start the full tour" or "show me everything".

Custom tour: Say "give me a small tour". The TSP GUI opens; select stops and click Start.

Stop: Say "stop" or "abort" at any time for immediate halt.

Chat: Any utterance that does not match a navigation intent returns a friendly conversational response.

31.2 Command / Voice Reference

Intent	Example Phrases
navigate	"take me to pillar 3", "go to the lab", "lead me to waypoint 2"
start_tour	"start the tour", "show me everything", "full tour please"
tsp	"small tour", "custom tour", "let me choose some stops"
dock	"go home", "recharge", "go to the dock"
stop	"stop", "halt", "freeze", "abort", "cancel"
follow_me	"follow me", "come with me", "walk with me"
explain	"what is this", "tell me about the lab", "describe this area"
is_alive	"hey pepper", "are you there", "are you awake"

31.3 Anti-Patterns

- Do not set `energy_threshold` above 300 in quiet environments — utterances will be clipped.
- Do not disable `require_wake_word` in noisy public spaces — background speech will trigger commands.
- Do not use Ollama with large models (13B+) on hardware without a GPU — latency exceeds 5s per intent.

32 Testing

32.1 Running the Test Suite

```
colcon test --packages-select turtlebot_llm_control
colcon test-result --verbose
```

32.2 Mock-Mode Integration Test

Run intent pipeline without microphone:

```
ros2 launch turtlebot_llm_control intent_only.launch.py \
  enable_microphone:=false
# Inject synthetic utterances
```

```
ros2 topic pub /speech/text std_msgs/msg/String \
  "data: 'take me to pillar 3'" --once
ros2 topic echo /speech/intent
```

32.3 What Is and Is Not Tested

LLM intent classification has been validated at 95% accuracy on 200 hand-crafted utterances in the demo environment. Faster-whisper transcription accuracy and Nav2 path following are validated through Gazebo simulation.

32.4 Manual Testing Checklist

"stop" produces intent: stop within 100 ms.

"take me to pillar 3" produces intent: navigate, location: pillar_3.

"follow me" produces intent: follow_me.

Unknown conversational input produces intent: no_action with a coherent response.

/done_talking is published only after TTS audio has completed.

33 Deployment

33.1 Build Process

```
colcon build --packages-select turtlebot_llm_control
source install/setup.bash
```

33.2 Launch Files

33.2.1 intent_only.launch.py — recommended for integration

Launches all SS3 nodes. Use this when the locomotion stack is already running separately.

```
ros2 launch turtlebot_llm_control intent_only.launch.py \
  enable_microphone:=true \
  llm_provider:=groq \
  llm_model:=llama-3.1-8b-instant \
  require_wake_word:=true
```

33.2.2 bringup.launch.py

Full SS3 system including BT orchestrator. Used for standalone HMI testing.

33.3 Post-Deployment Verification

```
ros2 topic list | grep speech
ros2 topic echo /speech/intent
```

33.4 Rollback

Remove or replace the api.key file to switch providers. Change llm_provider and llm_model in the launch arguments.

34 Troubleshooting and FAQs

34.1 Common Errors

Symptom	Cause / Fix
No audio captured	Wrong <code>mic_device_index</code> . List devices with <code>python3 -c "import speech_recognition as sr; print(sr.Microphone.list_microphone_names())"</code> .
LLM disabled at startup	API key not found. Set <code>GROQ_API_KEY</code> or <code>OPENAI_API_KEY</code> , or use <code>llm_provider:=ollama</code> .
<code>/done_talking</code> never arrives	TTS command not installed. Install <code>spd-say</code> or <code>espeak</code> .
High LLM latency	Using large Ollama model without GPU. Switch to <code>llm_provider:=groq</code> .
Wake word not activating	<code>energy_threshold</code> too high. Lower to 100 and re-test.

34.2 Useful Debug Commands

```
ros2 topic echo /speech/text
ros2 topic echo /speech/intent
ros2 topic echo /done_talking
ros2 topic echo /speech/tts_active
```

34.3 Log Locations

```
~/ros/log/latest/ # ROS2 node logs
```

34.4 FAQs

How do I switch LLM providers? Pass `llm_provider:=groq` and `llm_model:=llama-3.1-8b-instant` as launch arguments. Set `GROQ_API_KEY` in the environment.

How do I add a new intent? Add a matching branch in `speech_parser.py`'s `parse_utterance()` function, add the intent name to `llm_dialogue.py`'s `allowed_intents` list, and add the handler branch to `bt_orchestrator_node.py`.

35 Maintenance and Contributing

35.1 Branching and Git Workflow

Feature branches off `main`, prefixed `feat/ss3/` or `fix/ss3/`.

35.2 Commit Message Format

```
<type>(ss3): <short description>
```

35.3 Code Style

PEP8, enforced via `flake8`. No inline comments unless the WHY is non-obvious.

35.4 Adding a New Node

Register in `setup.py` `console_scripts`, add to `intent_only.launch.py`, document topic table in this section.

35.5 Adding a New Intent / Feature

Three-step process: add to `parse_utterance()`, add to `allowed_intents`, add BT handler. See the `follow_me` intent as a reference implementation.

35.6 Updating Dependencies

Update `package.xml` and `setup.cfg`, rebuild with `colcon build`.

35.7 Versioning

SemVer. MAJOR for IntentToken field changes, MINOR for new intents, PATCH for parser/LLM tuning.

36 Security

36.1 Credential Management

Warning: Never commit `api.key` or any file containing an API key. The `.gitignore` excludes `api.key` but this must be verified after any new file additions.

Prefer environment variables (`GROQ_API_KEY`, `OPENAI_API_KEY`) over the key file.

36.2 Known Security Considerations

The `/speech/text` topic is unauthenticated. A malicious publisher could inject arbitrary intents. Restrict topic access in production via DDS security.

36.3 Vulnerability Reporting

Report issues via the GitHub repository issue tracker.

37 Changelog and Versioning

37.1 Version History

Version	Date	Changes
0.1.0	May 2026	Initial release. Full STT/LLM pipeline, BT orchestrator, 18 intents, waypoint speaker, emotion publisher, TSP GUI. Breaking: <code>emotion_node</code> now publishes to <code>/pepper/gesture_command</code> (was <code>/emotions</code>). Added: <code>/pepper_speech</code> topic for Pepper TTS forwarding. Added: <code>follow_me</code> intent for QR-code-based person following. Fixed: <code>done_talking</code> now published only after TTS audio completes (was immediate). Added: Gesture "explain" published by <code>waypoint_speaker_node</code> before TTS begins.

37.2 Migration Guide — emotion topic rename

If any node subscribed to `/emotions`, update it to subscribe to `/pepper/gesture_command`.

37.3 Migration Guide — done_talking timing fix

If downstream nodes assumed `/done_talking` arrived immediately after speech was triggered, update them to handle the deferred signal. The signal now arrives only after TTS audio has completed.

Part IV

Subsystem 4 — Pepper Integration

38 Project Overview

38.1 Purpose

Subsystem 4 bridges the ROS2 ecosystem to Pepper's NAOqi hardware. It connects TTS commands from the ROS2 network to Pepper's `ALTextToSpeech` service, handles hardware safety and stiffness management, and enables Pepper's on-board autonomous navigation when required. It shares the `pepper_hri` package with Subsystem 2, with SS4 owning the hardware-integration layer.

38.2 Target Audience

Engineers responsible for Pepper hardware deployment, NAOqi SDK integration, and ROS2-NAOqi bridging.

38.3 Scope

What Subsystem 4 does:

- Bridges `/pepper/spoken_words` (ROS2 String) to NAOqi `ALTextToSpeech`.
- Manages Pepper motor stiffness (wakeUp / rest) on startup and shutdown.
- Provides NAOqi session management with reconnection on network interruption.
- Exposes the `audio_processor_node` for routing Pepper's onboard microphone to `/audio/audio`.
- Supports Pepper's autonomous navigation via NAOqi `ALNavigation` when enabled.

What it does NOT do:

- Manage TurtleBot3 navigation — that is SS1's responsibility.
- Classify speech intents — that is SS3's responsibility.
- Define or execute gesture sequences — that is SS2's responsibility.

38.4 Technology Stack

NAOqi SDK 2.5, qi Python bindings, ROS2 Humble, Python threading.

38.5 High-Level Architecture

ROS2 side		NAOqi side (Pepper)
-----		-----
<code>/pepper/spoken_words</code>	-->	<code>ALTextToSpeech.say()</code>
<code>/pepper/gesture_cmd</code>	-->	<code>ALMotion.angleInterpolation()</code>
<code>/audio/audio</code>	<--	<code>ALAudioDevice (mic stream)</code>
<code>/current_tour</code>	-->	<code>HRI coordinator exhibit display</code>

38.6 Related Projects

- `pepper_hri` package (shared with SS2) — gesture execution and HRI coordination.
- `turtlebot_llm_control` (SS3) — publishes `/pepper/spoken_words` and `/pepper/gesture_command`.

39 Getting Started

39.1 Prerequisites

- Pepper robot with NAOqi 2.5, powered on, and connected to the same LAN as the ROS2 host.
- qi Python SDK installed on the ROS2 host.
- Pepper's IP address known (default 192.168.1.100).

39.2 System Requirements

- Python 3.6 or later (qi SDK constraint).
- LAN connectivity between ROS2 host and Pepper (wired or reliable Wi-Fi).

39.3 Installation

39.3.1 Step 1 — Obtain qi SDK

```
pip install qi
# If qi is not on PyPI for your platform, download the NAOqi Python
  SDK
# from the SoftBank Robotics developer portal and install manually.
```

39.3.2 Step 2 — Configure Pepper IP

Edit `pepper_real.yaml` (in the `pepper_hri` config directory) and set `pepper_ip` to Pepper's LAN address.

39.3.3 Step 3 — Build

```
colcon build --packages-select pepper_hri
source install/setup.bash
```

39.4 Environment Variables

No environment variables required. All connectivity parameters are set via ROS2 node parameters.

39.5 Verify the Installation

```
python3 -c "
import qi
s = qi.Session()
s.connect('tcp://192.168.1.100:9559')
print('Connected to Pepper NAOqi')
"
```

39.6 Quick Start

```
ros2 launch pepper_hri pepper_real.launch.py \
  start_gestures:=true \
  start_hri:=true \
  start_audio:=false
```

40 Configuration

40.1 Launch Arguments — `pepper_real.launch.py`

Same as Subsystem 2 — see Part II, Section 3.1.

40.2 Node Parameters

40.2.1 `gesture_manager` (hardware bridge)

Same parameters as listed in Part II, Section 3.2. The `gesture_manager` node also serves as the NAOqi motion bridge for SS4.

41 API and Code Reference

41.1 Custom Interfaces

No new custom interfaces defined in SS4.

41.2 Data Models

The `audio_processor_node` captures Pepper’s microphone stream and publishes raw 8-bit unsigned audio samples as `UInt8MultiArray` on `/audio/audio`. The amplitude is computed as:

```
audio_samples = (raw_data.astype(np.int16) - 128) * 256
amplitude = np.max(np.abs(audio_samples))
```

41.3 Key Functions

`GestureManager.connect_to_pepper()` establishes a `qi.Session`, retrieves `ALMotion` and `ALRobotPosture` services, and calls `motion.wakeUp()` if `enable_stiffness` is set.

`GestureManager.start_gesture(command)` retries `connect_to_pepper()` if `ALMotion` is `None`, allowing the node to recover from Pepper reboots or network drops without restarting the ROS2 node.

42 Architecture and Design

42.1 Folder Structure

The hardware bridge lives in the same `pepper_hri` package as SS2:

```
src/Pepper_HRI/
  gestures/
    gesture_manager.py      % NAOqi motion bridge (SS4 owns)
  audio/
    audio_processor_node.py % Pepper mic -> /audio/audio (SS4 owns)
  HRI_coordinator.py        % Exhibit coordination (SS2/SS4 shared)
  pepper_real.launch.py     % Hardware launch entry point
```

42.2 Node Responsibilities and Topics

42.2.1 `gesture_manager` (hardware layer)

See Part II Section 5.2. In addition to executing gestures, `gesture_manager` is the sole node that holds an active NAOqi session and `ALMotion` handle.

42.2.2 `audio_processor_node`

Dir	Topic	Type	Notes
pub	/audio/audio	UInt8MultiArray	Raw PCM samples from Pepper mic

42.3 Key Design Decisions

Shared package, split ownership. SS2 and SS4 share `pepper_hri` to avoid duplicating NAOqi session management code. SS4 owns the hardware connectivity layer (`gesture_manager.py` NAOqi session, `audio_processor_node.py`); SS2 owns the HRI coordination logic (`HRI_coordinator.py`).

Connection retry on failure. Rather than crashing, `gesture_manager` stores the last connection error and retries automatically on the next gesture command. This allows deployment scripts to start the ROS2 node before Pepper has finished booting.

Separate audio node. The audio processor is disabled by default (`start_audio:=false`) because it requires manual IP configuration in the source file. This prevents silent failures during simulation.

43 Usage Guide

43.1 Common Use Cases

Hardware bring-up:

```
ros2 launch pepper_hri pepper_real.launch.py \
  start_gestures:=true start_hri:=true
```

Test TTS directly:

```
ros2 topic pub /pepper/spoken_words std_msgs/msg/String \
  "data: 'Hello, I am Pepper.'" --once
```

Enable Pepper microphone bridge: Edit `audio_processor_node.py` with Pepper's IP, rebuild, then:

```
ros2 launch pepper_hri pepper_real.launch.py start_audio:=true
```

43.2 Command / Voice Reference

Hardware integration is transparent to voice commands. All voice commands flow through SS3; SS4 receives the resulting gesture and speech topics.

43.3 Anti-Patterns

- Do not set `enable_stiffness:=false` if Pepper is standing — the robot will collapse when motors are released.

- Do not enable audio without configuring the IP in `audio_processor_node.py` — the node will fail silently.
- Do not run multiple `gesture_manager` instances pointing at the same Pepper — concurrent `ALMotion` calls conflict.

44 Testing

44.1 Running the Test Suite

No automated tests for NAOqi hardware calls. Manual verification is required.

44.2 Mock-Mode Integration Test

Use `gesture_manager_sim.py` to test the ROS2 integration without Pepper hardware. It accepts the same topic and parameters but logs instead of calling NAOqi.

44.3 What Is and Is Not Tested

The NAOqi session establishment, gesture execution, and TTS bridging are validated only on physical Pepper hardware. Network reconnection has been tested by pulling the Ethernet cable during operation.

44.4 Manual Testing Checklist

NAOqi session established within 5 s of node startup.

Pepper speaks correctly on `/pepper/spoken_words` publication.

Pepper raises arm on "wave_hello" gesture command.

Node reconnects after Pepper reboot without requiring ROS2 node restart.

Audio amplitude published on `/audio/audio` when speech is present.

45 Deployment

45.1 Build Process

```
colcon build --packages-select pepper_hri
source install/setup.bash
```

45.2 Launch Files

45.2.1 `pepper_real.launch.py` — recommended for hardware deployment

```
ros2 launch pepper_hri pepper_real.launch.py \
  start_gestures:=true \
  start_hri:=true \
  start_tablet:=true \
  start_audio:=false
```

45.2.2 pepper_sim.launch.py — simulation without Pepper hardware

```
ros2 launch pepper_hri pepper_sim.launch.py
```

45.3 Post-Deployment Verification

```
ros2 node list | grep -E "gesture|hri|audio"
ros2 topic echo /pepper/gesture_command --once
```

45.4 Rollback

Restart the `gesture_manager` node. Pepper will return to its current posture. Call `ALMotion.rest()` manually via NAOqi if Pepper needs to be put in rest mode.

46 Troubleshooting and FAQs

46.1 Common Errors

Symptom	Cause / Fix
Could not connect to Pepper NAOqi	Pepper not on LAN or wrong IP. Verify with <code>ping 192.168.1.100</code> .
Gesture command ignored silently	<code>gesture_manager</code> could not reconnect. Check NAOqi logs on Pepper.
No audio on <code>/audio/audio</code>	Audio processor not started or wrong IP configured. Enable <code>start_audio:=true</code> after IP is set.
Stiffness error on startup	Pepper already awake; call <code>ALMotion.wakeUp()</code> idempotently.

46.2 Useful Debug Commands

```
ping 192.168.1.100
ros2 topic hz /audio/audio
ros2 node info /pepper_gesture_node
```

46.3 Log Locations

```
~/ros/log/latest/      # ROS2 node logs
/var/log/naoqi/        # NAOqi logs on Pepper (via SSH)
```

46.4 FAQs

How do I change Pepper's IP? Update the `pepper_ip` parameter in `pepper_real.yaml` or pass it as a parameter override at launch.

What if qi SDK is not available on my platform? Use the simulated gesture manager (`gesture_manager_sim.py`) which has no NAOqi dependency.

47 Maintenance and Contributing

47.1 Branching and Git Workflow

Feature branches off main, prefixed `feat/ss4/` or `fix/ss4/`.

47.2 Commit Message Format

`<type>(ss4): <short description>`

47.3 Code Style

PEP8, enforced via `flake8`.

47.4 Adding a New Node

Register in `setup.py`, add to `pepper_real.launch.py`, document topic table in this section.

47.5 Adding a New Intent / Feature

NAOqi service calls (TTS, navigation, sensors) can be added to `gesture_manager.py` or as new nodes in the `pepper_hri` package.

47.6 Updating Dependencies

Update `package.xml` with any new NAOqi service dependencies.

47.7 Versioning

Follow SemVer. MAJOR for breaking NAOqi API changes, MINOR for new services, PATCH for connection stability fixes.

48 Security

48.1 Credential Management

No credentials required. NAOqi connects without authentication on the local network.

48.2 Known Security Considerations

NAOqi uses unauthenticated TCP on port 9559. Do not expose Pepper's NAOqi port to untrusted networks. The `/pepper/spoken_words` topic is unauthenticated and could be used to make Pepper say arbitrary text.

48.3 Vulnerability Reporting

Report issues via the GitHub repository issue tracker.

49 Changelog and Versioning

49.1 Version History

Version	Date	Changes
0.1.0	May 2026	Initial release. NAOqi gesture bridge, audio processor, hardware safety management, connection retry logic. Added: Automatic NAOqi reconnection on gesture command when session is lost. Added: <code>gesture_manager_sim.py</code> as hardware-free drop-in for testing.

A Complete Topic Reference

Topic	Type	Dir	Publisher	Subscriber(s)
/speech/text	String	pub/sub	speech_to_text_node	speech_command_node
/speech/intent	String	pub/sub	speech_command_node	bt_orchestrator_node, speech_listener, tour_intent_bridge
/speech/response	String	pub/sub	speech_command_node	speech_response_node
/speech/tts_active	Bool	pub	speech_response_node	
/pepper_speech	String	pub/sub	speech_response_node	Pepper HRI
/done_talking	String	pub/sub	speech_response_node	hri_coordinator_node, waypoint_speaker_node
/done_speaking	String	pub	waypoint_speaker_node	bt_orchestrator_node
/talk_command	String	pub/sub	bt_orchestrator_node	waypoint_speaker_node, tour_manager
/tour_command	String	pub/sub	speech_listener, bt_orchestrator	Tour controller
/tsp_command	TspCommand	pub/sub	speech_listener	TSP controller
/dock_command	String	pub/sub	speech_listener	dock_listener
/save_dock_command	String	pub/sub	—	dock_listener
/follow_command	String	pub/sub	speech_listener	qr_follower
save_tour	PoseStamped	pub/sub	Route recorder	tour_manager
current_tour	String	pub/sub	tour_manager	hri_coordinator_node
/pepper/gesture_command	String	pub/sub	waypoint_speaker_node	gesture_manager emotion_node, hri_coordinator
/pepper/spoken_words	String	pub/sub	hri_coordinator_node	Pepper TTS bridge
/pepper/explain_exhibit	String	pub/sub	hri_coordinator_node	Tablet display node
/robot/emotion	String	pub/sub	waypoint_speaker_node	emotion_node
/human_present	String	pub/sub	Human detector	hri_coordinator_node
/audio/audio	UInt8MultiArray	pub/sub	audio_processor_node	hri_coordinator_node
/qr_follower/status	String	pub	qr_follower	—

Topic	Type	Dir	Publisher	Subscriber(s)
/cmd_vel	Twist	pub/sub	qr_follower	TurtleBot3 base
/amcl_pose	PoseWithCovStamped	pub/sub	AMCL	dock_listener